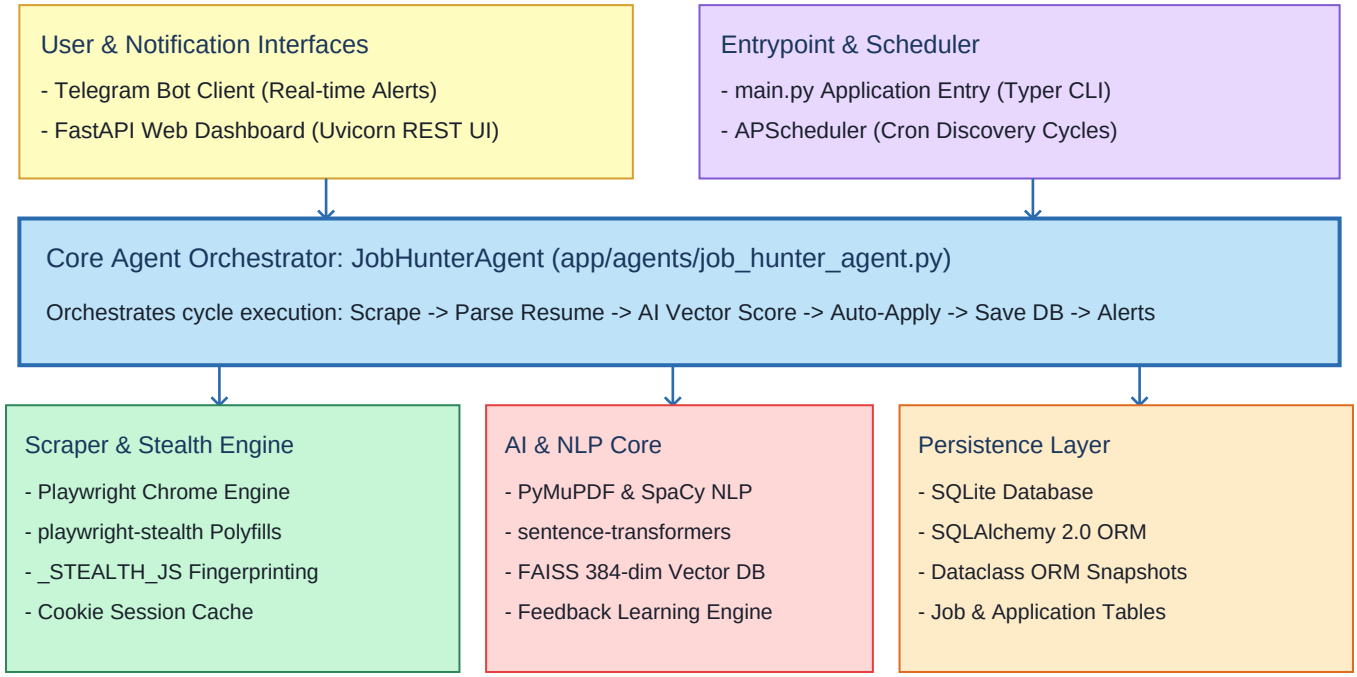
Naukribot - System Architecture & Workflow

Production-Grade Autonomous Job Discovery, AI Vector Matching & Auto-Apply System

1. High-Level System Architecture Diagram



2. Core Technology Stack

Web Automation	Playwright (Python Async API), playwright-stealth, BeautifulSoup4
AI Vector Embeddings	sentence-transformers/all-MiniLM-L6-v2 (384-dim), FAISS vector store
NLP & Resume Parsing	PyMuPDF (fitz), spaCy (en_core_web_sm), Skill Taxonomy Matcher
Backend & API Server	FastAPI, Uvicorn (ASGI), Pydantic v2, Pydantic-Settings
Database & ORM	SQLite 3, SQLAlchemy 2.0 (ORM Session-safe Dataclasses)
Scheduler & Bot	APScheduler (Background Cron), python-telegram-bot (v21 async)

3. Operational Features Summary

- Chrome Stealth Scraping: Uses real installed Chrome with JS polyfills overriding navigator.webdriver, permissions, and browser flags to pass Naukri anti-bot checks.
- Session Cookie Cache: Saves login cookies to data/cache/naukri_session.json. Automatically restores sessions without repeating manual logins.
- Semantic Vector Match: Embeds candidate resume and job descriptions into a 384-dimensional FAISS index for high-precision semantic matching.
- Chatbot Questionnaire Solver: Intercepts post-apply recruiter screening popups during auto-apply and answers questions based on profile tags.
- Telegram Bot & Dashboard: Sends instant notifications with apply buttons to Telegram and exposes real-time analytics on http://localhost:8000.

4. System Execution Workflow (25-Step Visual Flowchart)

PHASE 1: System Bootstrap & Initialization

- 1. Launch application via main.py (CLI / Background)
- 2. Initialize SQLite tables via SQLAlchemy ORM (init_db())
- 3. Parse candidate resume text & extract skills (PyMuPDF / SpaCy)
- 4. Pre-load FAISS vector embedding model (SentenceTransformers)
- 5. Start background APScheduler, Telegram bot & FastAPI server

PHASE 2: Playwright Stealth Search & Discovery

- 6. Trigger discovery cycle every N minutes via APScheduler
- 7. Launch Playwright browser using real Chrome channel
- 8. Inject stealth JS scripts to override webdriver & browser flags
- 9. Restore authenticated session cookies from data/cache
- 10. Execute search matrix (Target Roles x Preferred Locations)
- 11. Extract raw job card metadata from search result pages

PHASE 3: Filtering & Database Persistence

- 12. Check external job ID against SQLite database (Deduplication)
- 13. Apply hard experience & location bounds filtering
- 14. Save valid new jobs to database as session-safe JobData objects

PHASE 4: AI Matching & Ranking Engine

- 15. Generate FAISS vector embedding for scraped job text
- 16. Compute cosine similarity between job & resume embeddings
- 17. Calculate skill fit, experience fit & posting freshness scores
- 18. Apply historical user feedback boost (Learning Engine)
- 19. Store final match score (0 - 100%) in SQLite database

PHASE 5: Auto-Apply & Real-Time Alerts

- 20. Filter roles meeting match threshold (Score >= Threshold / Entry)
- 21. Navigate to job application URL via Playwright
- 22. Verify direct 1-Click apply eligibility
- 23. Click Apply & solve recruiter chatbot forms (ChatbotHandler)
- 24. Update application status to 'Applied' in SQLite DB
- 25. Dispatch instant Telegram notification & refresh Web Dashboard

5. Strategies for Uncovering Hidden Jobs on Naukri

Naukri.com search caps results at Page 50 (1,000 jobs). Roles past page 50 or in candidate feeds become hidden.

1. 24-Hour Freshness Filtering (freshness=1)

Over 80% of applicants apply within 48 hours. Appending freshness=1 to search URLs isolates jobs posted in the last 24h, bypassing page 50 pagination cutoffs.

2. Exact Boolean Query Parameters (qp)

Standard UI search uses loose keyword matching. Using explicit URL parameters (qp="Machine Learning" AND ("PyTorch" OR "TensorFlow") NOT "Intern") exposes precise unlisted job postings.

3. Authenticated Recommendation Feed Scraping

Naukri maintains an unindexed matching feed at /mnjuser/recommendedjobs. Naukribot scrapes this feed post-login to access personalized job recommendations.

4. Daily Profile Activity Refresh

Recruiters filter candidate searches by 'Active in last 7 days'. Re-uploading your resume daily touches your lastUpdatedTimestamp, bringing unlisted recruiter outbound invites.

5. Google X-Ray Search Queries (Google Dorks)

Recruiter landing pages are indexed by Google before appearing in search. Queries like site:naukri.com/job-listings "Machine Learning Engineer" uncover hidden direct URLs.

6. Scraper Stealth & Anti-Bot Evasion Specifications

```
// _STEALTH_JS Script Injected into Every Playwright Page Context:
1. navigator.webdriver Override  : Object.defineProperty(navigator, 'webdriver', { get: () => undefined });
2. Chrome Runtime Masking        : window.chrome = { runtime: {} };
3. Plugin Array Spoofing         : Object.defineProperty(navigator, 'plugins', { get: () => [1, 2, 3, 4, 5] });
4. Language & Platform Spoofing  : Languages: ['en-US', 'en'], Platform: 'Win32'
5. Permission Interception      : Intercepts navigator.permissions.query for notifications

Chromium Launch Arguments:
- --disable-blink-features=AutomationControlled
- --no-sandbox   - --disable-infobars   - --start-maximized
```

7. AI Matching & Ranking Formula

Final Match Score Formula (0 - 100%):

```
Score = (Vector Similarity * 0.40) + (Skill Fit * 0.20) + (Exp Match * 0.15) + (Loc Match * 0.15) + (Boost * 0.10
```

- Vector Similarity (40%): Cosine distance between resume embedding and job description embedding via FAISS.
- Skill Fit (20%): Direct overlap count between candidate skill taxonomy and required job skill tags.
- Experience & Location (30%): Range overlap between job requirements and candidate profile constraints.
- Learning Boost (10%): Adjustment score learned by LearningEngine based on historical apply/ignore user actions.

8. SQLite Database Schema & Data Model

Job Table (jobs)

`id`	INTEGER (Primary Key)
`external_id`	VARCHAR (Unique Naukri Job ID)
`title / company`	VARCHAR (Job Title & Hiring Company)
`location / salary`	VARCHAR (Job Location & Compensation)
`experience_min / max`	FLOAT (Min and Max Experience Years)
`required_skills`	TEXT (JSON Array of Skill Tags)
`apply_url`	VARCHAR (Direct Apply or Detail Link)
`final_score`	FLOAT (Calculated AI Match Score 0-100)
`status`	VARCHAR ('new', 'applied', 'rejected', 'saved')

JobApplication Table (job_applications)

`id`	INTEGER (Primary Key)
`job_id`	INTEGER (Foreign Key -> jobs.id)
`applied_at`	DATETIME (Application Submission Timestamp)
`match_score`	FLOAT (Score at Time of Application)
`status`	VARCHAR ('applied', 'failed', 'pending')
`notes`	TEXT (Auto-apply Execution Log / Notes)

CandidateProfile Table (candidate_profiles)

`id`	INTEGER (Primary Key)
`name / email / phone`	VARCHAR (Candidate Contact Info)
`headline / summary`	TEXT (Resume Headline & Profile Summary)
`total_experience_years`	FLOAT (Calculated Experience Years)
`resume_text`	TEXT (Extracted Plain Resume Text)

9. Configuration & Deployment Setup

```
Configuration Settings (config/config.yaml):

search:
  target_roles: ['Machine Learning Engineer', 'Full Stack Developer']
  locations: ['Remote', 'Hyderabad', 'Pune']
  experience_min: 0
  experience_max: 3

naukri:
  email: 'your_email@example.com'
  password: 'your_password'
  headless: false               # Set true for background docker runs
  max_jobs_per_cycle: 20

ranking:
  alert_threshold: 65.0         # Score threshold for auto-apply & alerts
```

10. Telegram Bot Interactive Commands

`/jobs`	Fetch latest high-match job listings scraped by the bot.
`/topjobs`	Retrieve top 20 ranked jobs sorted by AI score.
`/newjobs`	Display jobs discovered within the last 1 hour.
`/stats`	Show application statistics, auto-apply counts, and success rates.
`/companies`	List top hiring companies extracted from active listings.
`/skillgaps`	View skill gap analysis identifying missing skills across scraped jobs.

System Status: Production Ready

Run Command: python main.py

Web Dashboard: http://localhost:8000 | Database: data/naukribot.db